

Product Requirements Document — Portfolio Dashboard

Product Requirements Document — Portfolio Dashboard

Product	Portfolio Dashboard — unified stock-portfolio analytics for Indian markets (NSE/BSE)
Document version	1.1
Status	Live (shipped)
Last updated	2026-06-23
Owner	Raaghav Pilaniwala
Repository	github.com/capraaghav/portfolio-dashboard
Tech stack	Python · Streamlit · pandas · yfinance · Plotly · pdfplumber · Supabase (optional)

1. Executive summary

Portfolio Dashboard is a **single-page web application that consolidates a retail investor's stock holdings from multiple Indian brokerage accounts into one unified, interactive analytics dashboard**. A user uploads one or more account exports (CSV, Excel, or PDF), and the app normalises wildly different broker formats into a single dataset, resolves each holding to its live NSE/BSE ticker, fetches live prices and fundamentals from Yahoo Finance, and presents valuation, allocation, technicals, analyst targets, tax, risk, dividends, per-stock detail, benchmarking, and rebalancing across eleven tabs.

It is **local-first and privacy-preserving** by design — holdings are processed on the user's own machine and never uploaded to a third party (the only outbound calls are anonymous market-data lookups). It can also be **hosted as a zero-install shared link**, in which case each visitor's data is isolated to their own session.

Optionally, the hosted deployment can be put behind **email-login user accounts with per-user cloud persistence** (Supabase Auth + Postgres with row-level security), so a user's portfolio, snapshots, watchlist, and overrides survive server reboots and follow them across devices — while remaining private to their account. This backend is **opt-in and additive**: with no Supabase configured the app behaves exactly as the local-first / per-session product described above.

The defining capability is the **ingestion & resolution engine**: it accepts the messy, inconsistent exports that Indian brokers actually produce — company names instead of tickers, ISIN-only rows, NSE series suffixes, multi-sheet workbooks, metadata headers, footers/disclaimers, and formatted PDF statements — and reliably turns them into clean, live-priced, consolidated holdings.

2. Problem statement

A typical Indian retail investor holds stocks across **several brokerage accounts** (e.g. Zerodha, HDFC Securities, Groww) and often across **multiple family members' demat accounts**. Each broker:

- Exports in a **different file format** (CSV, Excel, PDF) with different column names, layouts, and number formatting.
- Identifies holdings inconsistently — some give clean tickers (RELIANCE), some give full company names ("ABB INDIA LIMITED EQ NEW RS. 2/-"), some give only an **ISIN**, some append **NSE series suffixes** (INDOWIND-BE).
- Provides **partial data** — some omit cost basis, some omit purchase dates, some include mutual funds and bonds mixed with equities.

As a result there is **no single place** where the investor can see their *true* consolidated position, total gain/loss, allocation, concentration risk, or tax exposure across all accounts. Existing portfolio trackers either (a) require linking broker credentials (a privacy/security concern for personal financial data), (b) charge a subscription, or (c) only support one broker. There is no free, private, multi-broker, multi-format tool that “just works” on the files these brokers actually produce.

3. Goals & non-goals

3.1 Goals

- **G1 — Universal ingestion.** Accept exports from any major Indian broker in CSV, Excel, or PDF, auto-detecting the format with no user configuration.
- **G2 — Reliable identity resolution.** Resolve every holding to its correct live NSE/BSE ticker, even from company names or ISINs, without mis-assigning.
- **G3 — One consolidated view.** Merge the same stock held across multiple accounts into a single position, while preserving the per-account breakdown.
- **G4 — Comprehensive analytics.** Deliver valuation, allocation, technical, fundamental, analyst, tax, risk, and dividend analysis in one place.
- **G5 — Privacy by default.** No holdings data leaves the user’s machine in local mode; isolated per-session in hosted mode.
- **G6 — Zero-friction operation.** One command (or one double-click) to run locally; one URL to use when hosted. No accounts, no API keys.

3.2 Success metrics

- $\geq 95\%$ of holdings in a typical multi-broker upload resolve to a live ticker.
- Consolidated totals reconcile to within rounding of the broker’s own stated totals.
- Zero unhandled exceptions across all tabs for any supported file.
- A non-technical user can go from file $\rightarrow$ dashboard in under 2 minutes.

3.3 Non-goals (explicitly out of scope)

- **No order execution / money movement.** The app never trades, transfers, or places orders. Rebalancing only *suggests*.
- **No broker API / credential linking.** Input is files only (privacy choice).
- **No real-time tick streaming.** Prices are close/last-traded snapshots, cached.
- **No multi-currency / non-Indian markets** (NSE/BSE focus; ₹).
- **No mutual-fund NAV engine / SIP tracking** (MFs are detected and shown at CSV value, not deeply analysed).
- **No multi-user collaboration / sharing.** Accounts (when enabled) are private and single-owner — no shared portfolios, teams, or social features.
- **No mandatory accounts.** User accounts and a cloud database are an **optional, opt-in** backend (§6.8); the app fully functions with no account and no backend.

4. Target users & personas

Persona	Description	Key needs
Primary — The multi-account retail investor	Holds stocks across 2–4 broker/demat accounts; moderately financially literate; privacy-conscious about financial data.	Consolidated value & P&L, allocation, tax exposure, “am I beating NIFTY?”
Secondary — The family-portfolio manager	Manages portfolios for family members across different brokers; wants a per-account split.	Combine all accounts, per-account drill-down, per-person breakdown
Tertiary — The shared-link recipient	A friend/relative the primary user shares the hosted app with; non-technical, on Windows.	Zero install, upload own file, see own dashboard privately

5. User stories

1. *As an investor*, I upload my Zerodha and HDFC exports together and see one consolidated portfolio with combined value and gain/loss.
 2. *As an investor*, when a stock is held in two accounts, I click its row and see how many shares and how much value sit in each account.
 3. *As an investor*, I upload an HDFC file that lists company names instead of tickers, and the app still prices everything live.
 4. *As an investor*, I upload an IIFL PDF statement and it extracts all my holdings.
 5. *As an investor*, I see my LTCG vs STCG split and an estimate of tax if I sold today.
 6. *As an investor*, I check whether my basket has beaten the NIFTY 50 over the past year.
 7. *As an investor*, I see which positions are over-concentrated and which sectors dominate.
 8. *As a privacy-conscious user*, I run the whole thing on my laptop and confirm no holdings data is uploaded anywhere.
 9. *As a sharer*, I send a friend a link; they open it on Windows, upload their own file, and never see my data nor I theirs.
-

6. Functional requirements

6.1 Data ingestion & normalisation

- **FR-1.1** Accept multiple files at once via drag-and-drop; each file = one account, with a user-editable account label.
- **FR-1.2** Support file types: **CSV, XLSX, XLS, PDF**.
- **FR-1.3** Auto-detect known broker formats by their column signatures: Zerodha Kite, Zerodha Console (incl. detailed Excel with Sector + Long-Term Qty), HDFC Securities, Reliance/IndusInd (depository style), Groww, Upstox, Angel One.
- **FR-1.4** For unknown formats, **fuzzy-match column headers** to a canonical schema (ticker, shares, avg_cost, ltp, current_value, pnl, isin, sector, qty_long_term, purchase_date) using an alias dictionary; normalise UPPER_SNAKE_CASE (NSE_SYMBOL, COST_PRICE, ISIN_CODE) by treating underscores as spaces.
- **FR-1.5 Excel**: auto-detect the correct sheet and the header row even when buried under metadata rows; handle multi-sheet workbooks.
- **FR-1.6 CSV**: auto-detect the header line under preamble/metadata rows.

- **FR-1.7 PDF:** extract holdings page-by-page from text-based portfolio statements (e.g. IIFL Portfolio+), detecting ALL-CAPS ticker rows, skipping sector headings / column headers / footers, and parsing Indian-comma numbers and parenthesised negatives. Reject scanned/image PDFs with a clear message.
- **FR-1.8** Parse Indian-formatted numbers: lakh/crore commas (1,09,912.73), parenthesised negatives ((-7,400.64)), ₹, %, and --/blank tokens.
- **FR-1.9** Drop junk rows: totals, blank rows, and footer/disclaimer paragraphs (length-capped).
- **FR-1.10** Strip NSE series suffixes from tickers (-BE, -BZ, -SM, -EQ...) without harming legitimately hyphenated symbols (BAJAJ-AUTO).
- **FR-1.11** Surface per-file parse errors without failing the whole upload.

6.2 Ticker identity resolution (the core differentiator)

A holding may arrive as a clean ticker, a full company name, or only an ISIN. The app resolves each to a bare NSE/BSE symbol through a **3-tier chain**, and **never mis-assigns** — an unresolved holding is kept at its CSV value rather than guessed.

- **FR-2.1 Clean tickers** pass through unchanged.
- **FR-2.2 Company names** are cleaned (strip LIMITED, EQ, face-value junk) and resolved via **Tier 1 — Yahoo Finance search**, disambiguated by matching the candidate's live price to the file's own LTP (so "ABB India" → ABB, not Abbott India; "SAIL EQUITY SHARES" → SAIL, not Sai Life Sciences).
- **FR-2.3 ISINs** are resolved authoritatively via the **NSE official symbol master** (Tier 2), falling back to Yahoo.
- **FR-2.4 Tier 2 — NSE official equity list** (fetched live) resolves names Yahoo misses by fuzzy company-name match, and is authoritative for ISINs (it knows current names, e.g. Adani Wilmar is now AWL).
- **FR-2.5 Tier 3 — broader web search (DuckDuckGo)** is the last resort for renamed companies; it only accepts a symbol whose **live price matches the file's price**, so it cannot assign a popular ticker to an obscure/suspended holding.
- **FR-2.6** Resolution is cached 24h. The UI reports how many names/ISINs were auto-matched and how many could not be (shown at CSV value, flagged).
- **FR-2.7** Detect non-equity instruments — **mutual funds** and **bonds/NCDs** — and value them from the file (no live equity data), with an "Equity only" toggle to exclude them entirely.
- **FR-2.8** Allow **manual price override** for any holding Yahoo can't price.

6.3 Consolidation

- **FR-3.1** Merge identical tickers across accounts into one position with total shares, weighted average cost, combined value, and aggregate gain/loss.
- **FR-3.2** Preserve the list of contributing accounts per position.
- **FR-3.3** Compute live current value = total shares × live price, falling back to the broker-reported value when no live price is available.

6.4 Market data (Yahoo Finance via yfinance)

- **FR-4.1 Live quotes** — last price + previous close (for day change), fetched in **one bulk request per exchange** to avoid rate limits; per-ticker fallback for stragglers; numeric fields coerced to finite floats.
- **FR-4.2 Metadata** (cached 1h, with retries) — sector, company name, analyst targets, and fundamentals (P/E, P/B, market cap, beta, ROE, 52-week range), all from a single .info call per stock; numeric fields coerced at the source.
- **FR-4.3 Technical history** — 200 days of closes, bulk-downloaded.
- **FR-4.4 Dividends** — trailing-12-month dividends + yield, per stock.
- **FR-4.5 Benchmark indices** — NIFTY 50, SENSEX, Bank Nifty, Midcap.
- **FR-4.6 Per-stock detail** — 1-year OHLC history and recent news, on demand.
- **FR-4.7** A **"Refresh data"** action clears all caches and re-fetches.

6.5 Dashboard — the eleven tabs

#	Tab	Requirements
1	 Overview	Treemap heatmap (size = value, colour = P&L, grouped by sector); allocation donuts by stock and by sector; per-account totals table + stacked bar (when >1 account). Sortable/searchable table; toggle fundamentals columns; export to Excel; manual price override; click a row → per-account split of that stock (shares, cost, value, P&L per account + TOTAL) for stocks held in 2+ accounts. Filters (combinable): sector, asset type
2	 Holdings	(Equity/Fund/Bond), technical trend signal, account, and a “held in all accounts only” toggle — each narrows the table to matching holdings only. Auto-saved daily snapshots → portfolio value timeline; XIRR (when purchase dates exist); benchmark backtest of the current basket vs a chosen index, with alpha.
3	 Performance	SMA 20/50/200 + RSI 14 → trend signal per stock (Strong Bull → Strong Bear); signal-count cards; vs-50MA and RSI bar charts.
4	 Technical	12-month consensus price targets (low/mean/high) with upside %, Buy/Hold/Sell consensus, # analysts; target-range chart; coverage count.
5	 Analysts	LTCG/STCG split (uses long-term-qty or purchase dates); estimated tax (India rates); tax-loss-harvesting candidates.
6	 Tax	Largest position %, top-5 weight, effective # holdings (1/HHI), portfolio beta, sector concentration; concentration warnings.
7	 Risk	TTM dividend income per stock, portfolio yield, income chart (requires dividend data toggle).
8	 Dividends	Per-stock candlestick (1y) with SMA + avg-cost line, 52-week gauge, fundamentals, analyst consensus, dividend history, recent news (live-priced equities only).
9	 Stock Detail	

#	Tab	Requirements
10	 Watchlist	Track non-owned tickers (live price, day change, analyst target); persisted.
11	 Rebalance	Editable target weights → drift and ₹ to buy/sell per holding; never executes.

6.6 Summary cards (always visible)

Total value, total gain/loss (₹ + %), today's change, # holdings, # accounts, live price coverage (X/Y). Gain/loss shows “—” when the file has no cost basis.

6.7 Persistence

- **FR-7.1** Auto-save a **daily snapshot** of total value + per-stock values.
- **FR-7.2** Remember the **last session** so the user can reload without re-uploading; the loaded session survives reruns (filtering/interaction does not drop it).
- **FR-7.3** Persist **watchlist** and **manual price overrides**.
- **FR-7.4 Local / per-session storage (default)**: all persistence under a local data/ folder; in hosted multi-user mode, an isolated per-session temp folder.
- **FR-7.5 Cloud storage (optional, when Supabase is configured)**: the same four artifacts (last session, snapshots, watchlist, overrides) are stored **per user in Supabase Postgres** instead of local files, keyed to the authenticated user and protected by row-level security, so they persist across reboots and devices. The storage layer is interface-compatible with the local one (db.py mirrors storage.py), so call sites are identical.

6.8 Accounts & authentication (optional backend)

Active only when SUPABASE_URL + SUPABASE_ANON_KEY secrets are present; otherwise the app runs with no login, exactly as the local-first product.

- **FR-8.1 Email login / sign-up gate** (Supabase Auth). When enabled, an unauthenticated visitor sees a branded login / sign-up screen and the dashboard is withheld until they authenticate.
- **FR-8.2 Per-user data isolation via row-level security** — each user can read and write only their own rows (auth.uid() = user_id); a second account sees none of the first's data, enforced in Postgres (not just the client).
- **FR-8.3 Stay logged in across a browser refresh** — the Supabase refresh token is held in a browser cookie and used to re-authenticate on load, so a hard refresh (which clears Streamlit session state) restores the session instead of forcing re-login. Tokens rotate on refresh; logout deletes the cookie and clears all per-user session state so the next user on a shared browser starts clean.
- **FR-8.4 Sidebar account controls** — the logged-in user's email and a **Log out** button; the privacy caption reflects the active storage mode.
- **FR-8.5 Graceful, additive design** — the anon key is safe client-side (RLS is the protection); if the backend is unconfigured or unreachable, the app falls back to local behaviour rather than failing.

6.9 Interface & feedback polish

- **FR-9.1 Skeleton loaders** — a dashboard-shaped placeholder (hero + KPI cards + chart block) renders during the first cold data load, replaced once holdings are built; skipped on warm reruns to avoid flicker.
- **FR-9.2 Click-spark effect** — a lightweight canvas animation emits gold sparks from each click across the app (ported from a React component to a Streamlit-injected canvas overlay that persists across reruns).

7. Non-functional requirements

- **NFR-1 Privacy.** In local mode, holdings never leave the machine; only anonymous market-data lookups (Yahoo/NSE/DuckDuckGo) go out. In hosted per-session mode (PORTFOLIO_MULTIUSER=1), each browser session gets an isolated temp directory so no visitor can see another's data. In **accounts mode (Supabase)**, holdings are stored in the user's own database rows, isolated by Postgres row-level security so no user can read another's data; secrets (URL/anon key) live only in Streamlit secrets and are never committed (the anon key is safe to expose — RLS is the control).
 - **NFR-2 Resilience to rate limits.** Bulk-download quotes/history; retry `.info` with backoff; gracefully degrade (fall back to Yahoo-only, or CSV value) if a data source is unreachable; never crash on a missing/odd field (numeric coercion + defensive formatters).
 - **NFR-3 Performance.** First load of a ~150-stock multi-broker portfolio completes in tens of seconds; subsequent interactions are instant via caching (5 min quotes, 1 h metadata/TA, 24 h resolution & NSE master).
 - **NFR-4 Robustness.** Zero unhandled exceptions across tabs for any supported file; per-file parse errors are surfaced, not fatal.
 - **NFR-5 Usability.** No configuration, accounts, or API keys; Indian number formatting (₹, lakh/crore); clear messaging for unresolved/un-priced/CSV-only rows.
 - **NFR-6 Portability.** Runs on macOS, Windows, Linux (Python 3.9–3.14) locally, and on Streamlit Community Cloud.
-

8. Technical architecture

Pattern: a single Streamlit app composed of focused, independently-testable Python modules. Default state is the uploaded file plus a local `data/` folder and Streamlit's per-session cache; an **optional Supabase backend** can swap in cloud accounts + per-user storage without changing call sites.

Module	Responsibility
<code>app.py</code>	UI orchestration: sidebar, data-loading pipeline, summary cards, the 11 tabs, hosting/multi-user gating, and the backend selector (<code>store = db if db.is_enabled() else storage</code>) + auth gate.
<code>parsers.py</code>	File reading + broker detection + normalisation to the canonical schema; CSV/Excel/PDF; company-name cleaning; ISIN/series-suffix handling.
<code>market_data.py</code>	All Yahoo Finance fetching (quotes, metadata, TA history, dividends, benchmarks, per-stock detail), the 3-tier resolution chain, and the NSE master fetch. Cached.
<code>analytics.py</code>	Pure (no-I/O) computation: holdings consolidation, totals, per-account breakdown, technical indicators, risk metrics, LTCG/STCG tax, XIRR, synthetic backtest curve, rebalancing.
<code>charts.py</code>	Reusable Plotly figure builders (treemap, donuts, candlestick, benchmark overlay, etc.).
<code>storage.py</code>	Local persistence (snapshots, last session, watchlist, overrides); configurable data directory for per-session

Module	Responsibility
	isolation.
db.py	Optional Supabase backend — email auth (sign-in/up/out, cookie-based session restore) + per-user cloud storage, mirroring storage.py's signatures so it is a drop-in alternative; per-session client; activates only when secrets are present.
click_spark.py	Decorative click-spark canvas overlay (JS injected via a Streamlit component, persists across reruns).
formatting.py	Indian ₹/%/number formatters (defensive: coerce-or—"") + shared colour/label constants.

Data flow: upload → parsers.parse_all → resolution (market_data.resolve_symbols) → rename to clean tickers → market_data.fetch_quotes (+ optional metadata/TA/dividends) → analytics.build_holdings → tabs render via charts + formatting.

External services: (*read-only, anonymous, always*) Yahoo Finance (yfinance + query2.finance.yahoo.com/v1/finance/search), NSE archives (archives.nseindia.com/.../EQUITY_L.csv), DuckDuckGo HTML search. (*authenticated, per-user, only when accounts are enabled*) Supabase Auth + Postgres REST.

Backend selection: at startup db.is_enabled() checks for Supabase secrets + the supabase package; if present the app gates on login and routes all persistence through db.py, otherwise it uses storage.py (local/per-session). The Supabase client is created **per session** (in st.session_state, never cache_resource, which would be shared across users); auth tokens are kept in session state and a refresh-token cookie.

Key dependencies: streamlit ≥1.35, pandas, numpy, yfinance, plotly, openpyxl, pyarrow, curl_cffi, pdfplumber; (*optional accounts*) supabase ≥2.0, extra-streamlit-components ≥0.1.71.

9. Data dictionary (canonical schema)

Field	Meaning	Source
ticker	NSE/BSE symbol (post-resolution)	broker file → resolution
shares	Quantity held	broker file
avg_cost	Purchase/average price per share	broker file (may be absent)
ltp	Last traded price from the file	broker file
current_value	Broker-reported market value	broker file (fallback)
pnl	Broker-reported P&L	broker file (optional)
isin	ISIN code	broker file (optional)
sector	Sector/industry	broker file or Yahoo
qty_long_term	Shares held >12 months	broker file (for tax)
purchase_date	Buy date	broker file (for XIRR)
account	Account label (= file)	user

10. Deployment & operations

- **Local:** `pip install -r requirements.txt` then `streamlit run app.py`, or double-click `run.command` (macOS) / `run.bat` (Windows). Opens at `localhost:8501`. Persistent `data/` folder.
 - **Hosted (shared link, no accounts):** push to GitHub → deploy on Streamlit Community Cloud, with secret `PORTFOLIO_MULTIUSER = "1"` to enable per-session isolation. Auto-rebuilds on each push.
 - **Hosted with accounts (optional):** create a Supabase project, run the schema (`user_state` + snapshots tables, each with RLS policies `auth.uid() = user_id`), and add `SUPABASE_URL` + `SUPABASE_ANON_KEY` to Streamlit secrets (and a gitignored local `.streamlit/secrets.toml` for dev). Once present, the app requires login for all visitors and stores data per user in Supabase. Secrets are never committed.
 - **Distribution:** a clean zip (`run.command/run.bat` + code, `data/` excluded) for recipients who prefer to run locally.
-

11. Limitations & known constraints

- **Market-data accuracy depends on Yahoo Finance** — prices are close/last-traded, not real-time ticks; coverage is best for large/mid-cap NSE stocks.
 - **Analyst targets are 12-month** consensus (no free 6-month source for India).
 - **Performance history** builds over time from daily snapshots; the benchmark backtest replays *today's* quantities (ignores when shares were actually bought).
 - **XIRR requires purchase dates**, which most holdings exports omit (tradebook needed).
 - **Tax figures are estimates, not advice** (India listed-equity: LTCG 12.5% above ₹1.25 L/yr, STCG 20%).
 - **PDF support is text-based only** — scanned/image statements need OCR (not built).
 - **Renamed-with-no-price / suspended / delisted** holdings may stay unresolved (by design — shown at CSV value, never mis-assigned).
 - **Hosted mode** depends on Streamlit Cloud's shared IPs, which NSE/DuckDuckGo may occasionally block; the app degrades gracefully to Yahoo-only.
-

12. Future roadmap (candidate enhancements)

- OCR for scanned PDF statements.
 - Tradebook ingestion for accurate XIRR + realised-gains history.
 - Mutual-fund NAV resolution (AMFI) and deeper MF analytics.
 - Alerts (price/target/rebalance-drift) and goal tracking.
 - A small curated rename/alias map for well-known corporate renames.
 - Correlation matrix / drawdown analytics on the Risk tab.
 - Auth enhancements for accounts mode: password reset, email verification flow, OAuth providers, and an optional “continue as guest” (no-account) path on the login screen.
-

13. Glossary

- **LTP** — Last Traded Price. **ISIN** — International Securities Identification Number. **LTCG/STCG** — Long/Short-Term Capital Gains. **XIRR** — money-weighted annualised return. **HHI** — Herfindahl-Hirschman concentration index. **NSE/BSE** — National / Bombay Stock Exchange. **NCD** — Non-Convertible Debenture. **SMA/RSI** — Simple Moving Average / Relative Strength Index. **RLS** — Row-Level Security (Postgres per-row access control). **Anon key** — Supabase's public client key, safe to expose because RLS enforces access.